

Nginx + Coraza WAF Security Dashboard

1) Project Purpose

This project is a practical web-application security lab and monitoring platform. It combines:

- A vulnerable application (attack target)
- An Nginx reverse proxy with Coraza WAF + OWASP CRS
- A backend service that parses WAF audit logs and enriches alerts with AI summaries
- A React dashboard that visualizes attacks, WAF state, and risk posture in near real time

Primary goals:

1. Detect and block malicious traffic (XSS, SQLi/path traversal patterns, request abuse).
2. Surface blocked/suspicious events on a live dashboard with severity and context.
3. Demonstrate an end-to-end cloud-ready architecture for WAF observability.

2) High-Level Architecture

Local / Single-Host Compose Topology

- **WAF (Nginx + Coraza)**: ``localhost:18080``
- **API Backend (Express)**: ``localhost:3000``
- **Frontend Dashboard (Vite + React)**: ``localhost:5173``
- **Vulnerable App**: internal container endpoint ``vulnerable-app:3000``

Request Path

1. Client sends request to ``http://localhost:18080``.
2. Nginx/Coraza inspects request and enforces rules.
3. If allowed, Nginx proxies to ``vulnerable-app:3000``.
4. Coraza writes JSON audit entries to ``logs/audit.log``.
5. API backend tails and parses this log, then exposes normalized alert data at ``/api/alerts``.
6. Frontend polls API every few seconds and renders attack feed + metrics.

3) Repository Components and Responsibilities

``waf/``

`waf/nginx.conf`

Core WAF gateway behavior:

- Loads Coraza Nginx module.
- Enables Coraza with ``coraza on;`` and custom rules file:
 - ``/opt/coraza/config/custom-coraza.conf``
- Adds anti-abuse controls:
 - ``limit_req_zone ... rate=5r/s``
 - ``limit_req ... burst=10 nodelay``
 - ``limit_req_status 429``
- Applies bot-style user-agent deny filter for selected tools.
- Proxies allowed traffic to the vulnerable app upstream.

`waf/coraza.conf`

Core Coraza behavior:

- ``SecRuleEngine On``
- Includes CRS setup and CRS rules:
 - ``/opt/coraza/config/crs-setup.conf``
 - ``/opt/coraza/owasp-crs/rules/*.conf``
- Adds custom explicit XSS rule:
 - ``SecRule ARGS "@rx <script>" ... deny,status:403``
- Enables JSON audit logging:
 - ``SecAuditEngine On``
 - ``SecAuditLogFormat JSON``
 - ``SecAuditLog /var/log/nginx/audit.log``
- Adds DoS visibility rule:
 - Logs response status ``429`` as:
 - ``"DoS Attack Prevented: High frequency of requests detected from IP"``

This gives dashboard visibility for rate-limited DoS attempts without changing frontend logic.

`api-backend/`

`api-backend/server.js`

Express service that bridges raw WAF logs and dashboard-ready alerts.

Key responsibilities:

1. **Log ingestion**

- Watches `AUDIT_LOG_PATH` for appended content.
- Reads only new bytes to avoid reprocessing entire file.
- Parses line-delimited JSON audit entries.

2. **Alert normalization**

- Extracts:
 - `timestamp`
 - `client_ip`
 - `uri`
 - `messages[]`
- Sets initial `ai_summary`.

3. **Severity signal**

- Marks `is_critical` when message text matches critical keywords
(for example: anomaly score exceeded, SQL injection, XSS, command injection).

4. **AI enrichment**

- Uses OpenAI SDK against OpenRouter (`baseUrl: https://openrouter.ai/api/v1`).
- Model: `openai/gpt-4o-mini`.
- Produces concise 1-2 sentence summaries per alert.
- Handles API errors and 429/rate-limit fallback messaging.

5. **Public API**

- `GET /api/alerts`: returns most recent alerts + count.
- `GET /api/waf/mode`: reads `SecRuleEngine` mode from Coraza config.
- `POST /api/waf/mode`: updates mode (`On` or `DetectionOnly`) and sends HUP signal to WAF container.

Environment variables:

- `OPENAI_API_KEY` (required for AI summaries)
- `AUDIT_LOG_PATH`
- `MAX_ALERTS`
- `WAF_CONFIG_PATH`
- `WAF_CONTAINER_NAME`

`frontend/`

`frontend/src/App.jsx`

Main dashboard app:

- Polls ``/api/alerts`` and ``/api/waf/mode``.
- Displays:
 - attack count
 - health status
 - computed savings metric
 - alert feed and details
 - attack surface/risk panel
- Supports WAF mode updates (``DetectionOnly`` vs ``On``) via backend endpoint.
- Uses severity classification buckets:
 - ``Critical``
 - ``Medium``
 - ``Low``
 - ``Non-critical``

Classification is based on ``is_critical``, rule messages, and URI/message signals.

``frontend/src/ComingSoon.jsx``

Placeholder component used for non-dashboard navigation sections.

``vulnerable-app/``

Intentional vulnerable target used for realistic WAF testing:

- SQL injection-prone login query composition.
- Reflected input endpoint for testing payload behavior.
- Simple storefront-like interface to emulate real traffic.

4) Data and Control Flow

A) Security Event Flow

1. Attack/browsing request hits WAF (``18080``).
2. Coraza/CRS evaluates request.
3. Request is blocked or forwarded.
4. Coraza emits JSON audit entry to ``logs/audit.log``.
5. Backend watcher parses new line and pushes alert in-memory.
6. Backend asynchronously enriches summary using OpenRouter.

7. Frontend polling receives updated alert list and re-renders.

B) WAF Mode Change Flow

1. User clicks mode button in frontend.
2. Frontend calls `POST /api/waf/mode``.
3. Backend edits `SecRuleEngine`` in mounted Coraza config file.
4. Backend sends `docker kill -s HUP waf`` to reload WAF.
5. Frontend polls and reflects new mode state.

5) Attack Categories and How They Are Triggered

Current observable categories in dashboard:

- **Non-critical**: normal requests with no significant threat signals.
- **Low**: weak suspicious signals (for example probe/test/telemetry-like URI markers).
- **Medium**: moderate indicators in URI/messages (for example union/select/script-like patterns without critical match).
- **Critical**: strong rule-message matches (XSS, SQLi, anomaly score exceeded, RCE indicators, etc.).

DoS throttling:

- High-frequency request bursts are limited by Nginx rate limiting.
- Throttled responses return `429``.
- Coraza logs these as explicit DoS prevention messages for dashboard visibility.

6) Infrastructure and Ports

Local Compose Ports

- WAF: `18080 -> 8080``
- API backend: `3000 -> 3000``
- Frontend: `5173 -> 5173``
- Vulnerable app: container-internal `3000``

Networks

- `public_net``: frontend and WAF exposure layer.
- `internal_net``: service-to-service path for WAF and backend dependencies.

Volumes

- `./logs:/var/log/nginx`` (WAF audit output)
- `./logs:/logs:ro`` (backend read-only log access)
- Coraza config mounts for both WAF runtime and backend mode management

7) EC2 Deployment Design (Split Hosts)

Deployment folder supports two-host strategy:

EC2-A (`deployment/docker-compose.ec2-a.yml``)

- Runs ``waf``, ``api-backend``, ``frontend``
- Uses ``deployment/nginx.prod.conf`` as WAF proxy config
- Mounts `../waf/coraza.conf`` into WAF and backend
- Passes ``OPENAI_API_KEY`` into backend environment

EC2-B (`deployment/docker-compose.ec2-b.yml``)

- Runs ``vulnerable-app`` only

``deployment/nginx.prod.conf``

- Mirrors local WAF policy (Coraza on, rate limiting, bot filtering)
- Proxies to EC2-B private address:
 - replace `<PRIVATE_IP_OF_EC2_B>` before deployment

8) Security Model and Project Value

This project demonstrates a practical layered defense pattern:

1. **Edge inspection and enforcement** (Nginx + Coraza + CRS)
2. **Traffic abuse control** (rate limiting + DoS audit visibility)
3. **Centralized security telemetry** (JSON audit logs)
4. **Human-readable triage support** (AI-generated summaries)
5. **Operational control plane** (runtime WAF mode toggle)
6. **Cloud-ready segmented topology** (edge host + protected asset host)

Use cases:

- Security demos and blue-team training
- WAF rule tuning exercises
- SOC-style dashboard prototyping
- Pre-production validation of attack detection pipelines

9) Operational Runbook (Quick)

Start local stack

```
docker compose up -d --build
```

Open UI/API

- Frontend: `http://localhost:5173`
- WAF endpoint: `http://localhost:18080`
- Alerts API: `http://localhost:3000/api/alerts`

Example test traffic

```
# Non-critical
curl -A "Mozilla/5.0" "http://localhost:18080/"

# Low
curl -A "Mozilla/5.0" "http://localhost:18080/telemetry-test"

# Medium
curl -A "Mozilla/5.0" "http://localhost:18080/union-select-test"

# Critical
curl -i -A "Mozilla/5.0" "http://localhost:18080/search?q=%3Cscript%3Ealert(1)%3C%2Fscript%3E"

# DoS burst
for i in $(seq 1 25); do
  curl -s -o /dev/null -w "%{http_code}\n" -A "Mozilla/5.0" "http://localhost:18080/rapid-dos-test"
done
```

10) Known Constraints and Future Enhancements

Current constraints:

- Alerts are held in-memory (`MAX_ALERTS` ring behavior), not persistent DB-backed events.
- AI enrichment depends on external API availability and quota.
- Severity mapping is heuristic; not a full SIEM taxonomy.

Recommended next steps:

1. Persist alerts to PostgreSQL/ClickHouse for history and analytics.
2. Add authentication/authorization on backend control endpoints.
3. Export metrics to Prometheus/Grafana.
4. Add automated regression tests for WAF policy and category mapping.
5. Add TLS termination and domain routing for production.

11) Summary

The system is an end-to-end WAF observability platform that detects and blocks attacks at the edge, translates security telemetry into actionable dashboard insights, and supports a split-EC2 deployment model suitable for production-style demonstrations.